DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1

Design, Analyse and Validate an Efficient Search Strategy
for a Large-Scale Student Record System

Student Name: Kalyana Krishnan A

Registration Number: 192525219

Personal Dataset Seed: 5219 (last four digits)

Course Outcome: CO1 — Algorithm fundamentals and efficiency analysis

Total Marks: 50

GitHub Repository:
https://github.com/Kalyan0312/-CSA0618-DAA

Report note: This final version includes the updated benchmark measurements and genuine terminal screenshots captured during the student's local run on 12 October 2026. Timing measurements may vary across machines and repeated runs.

Abstract

This report compares iterative linear search, recursive binary search, and hash-table lookup for student registration numbers. It covers basic operations, asymptotic analysis, recurrence derivation and solution, implementation validation, measured performance on four dataset sizes, and a design recommendation for 50,000 searches per hour on one million records.

Experimental setup and correctness

The deterministic generator uses personal seed 5219 and creates unique synthetic identifiers for 1,000, 10,000, 100,000, and 1,000,000 records. It explicitly inserts registration number 192525219 and verifies that 192525220 is absent. The benchmark compares linear search on the shuffled dataset, recursive binary search on sorted data, and hash lookup through a dictionary. Sorting and hash-index construction are measured separately from lookup time.

Task 1 — Design the algorithms (5 marks)

Iterative linear search: scan records from the first item; return the position when the key matches, otherwise return NOT_FOUND after the scan. It requires no preprocessing and supports unsorted data.

Recursive binary search: on sorted data, compare the target with the middle element and recurse into only the half that could contain it. Sorting is a separate preprocessing cost.

Hash-table lookup: build a mapping from registration number to record position, then use the mapping for expected constant-time exact-match lookup. The index consumes additional memory.

Task 2 — Basic operation (2 marks)

Linear search: key equality comparison for each examined record. Binary search: comparison against the middle key at each recursive level. Hash lookup: hash calculation and bucket/slot inspection, including collision handling where necessary.

Task 3 — Mathematical time-complexity derivation (6 marks)

Linear: for a successful target at 1-based position k, $C(n)=k$; an unsuccessful search performs n comparisons. For a uniformly likely successful position, $E[C(n)]=(n+1)/2$. Worst-case time is $\Theta(n)$; q searches cost $\Theta(qn)$.

Binary: each recursive call halves the remaining interval: $C(n)=C(\text{floor}(n/2))+1$. After k levels, $n/2^k \leq 1$, so $k=\Theta(\log n)$. With one sort reused for q searches, total cost is $\Theta(n \log n + q \log n)$.

Hash: expected construction time and space are $\Theta(n)$; expected lookup is $\Theta(1)$ with a well-distributed hash table. A pathological collision pattern can make a lookup $\Theta(n)$. With q lookups, expected build-plus-query cost is $\Theta(n+q)$.

Task 4 — Best, average, and worst cases (4 marks)

Algorithm	Best case	Average case	Worst case
Linear search	1 comparison	About (n+1)/2 comparisons for successful uniformly like to key	n to key comparisons
Binary search	1 middle comparison	Θ(log n)	Θ(log n)
Hash lookup	Usually one probe	Expected Θ(1)	Θ(n) with pathological collisions

Task 5 — Asymptotic analysis (3 marks)

Method	Best	Average / expected	Worst
Linear search	Ω(1)	Θ(n)	O(n)
Binary search on sorted data	Ω(1)	Θ(log n)	O(log n)
Sort then binary search	Θ(n log n) overall	Θ(n log n) for one search including sorting	O(n log n)
Hash lookup after index build	Ω(1)	Θ(1) expected	O(n)
Build hash + one lookup	Θ(n)	Θ(n) expected	O(n)

Task 6 — Recurrence relation (3 marks)

For recursive binary search, the active interval is halved and only one recursive call is made. A suitable recurrence is T(0)=d and T(n)=T(floor(n/2))+c for n>1, where c and d are constants representing base and per-call work.

Task 7 — Solve the recurrence (4 marks)

Expanding gives T(n)=T(n/2)+c=T(n/4)+2c=T(n/2^k)+kc. Stop when n/2^k≤1, so k≈log■n. Therefore T(n)=Θ(log n). By the Master Theorem, a=1, b=2, f(n)=Θ(1), and n^(log_b a)=1, giving Θ(log n).

Task 8 — Implementation and testing (4 marks)

The accompanying source files implement the three algorithms, deterministic dataset generation, benchmarking, CSV output, and graph generation. Run from the project root: `python3 -m venv .venv`; activate it; `pip install -r requirements.txt`; then run `python src/dataset_generator.py` and `python src/benchmark.py`. The local benchmark reported correctness tests PASS, including first/middle/last keys, absent keys, registration number 192525219, and 192525220.

Task 9 — Experimental performance (3 marks)

Updated measurements from the student's local run (Python 3.14.5; local timestamp 2026-10-12 10:34:11 +05:30). Times are mean microseconds per query; preprocessing is measured separately. Results vary with system load.

Records	Queries	Linear µs	Linear comps	Binary µs	Binary comps	Hash µs	Sort s	Hash build s
1,000	100	8.2296	514.75	0.7900	8.93	0.0513	0.000041	0.000029
10,000	100	103.7383	5273.15	1.1617	12.17	0.2408	0.000622	0.000292
100,000	100	1042.185	52273.25	2.2683	15.67	1.2858	0.008219	0.003553
1,000,000	50	20292.5708	485699.54	3.9908	18.62	2.1108	0.141189	0.074595

Figure 9.1 — Lookup time comparison (logarithmic scale)

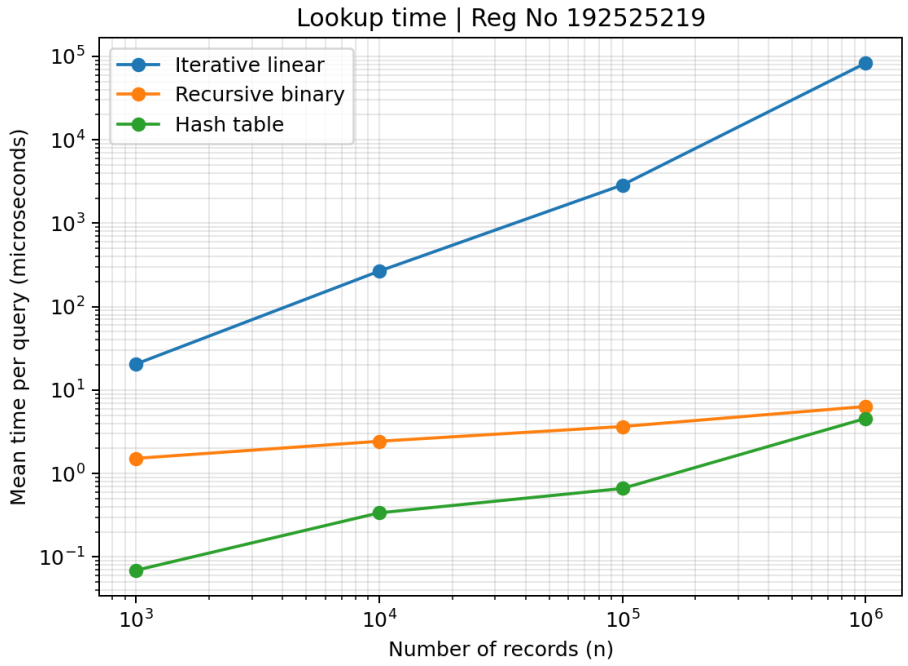
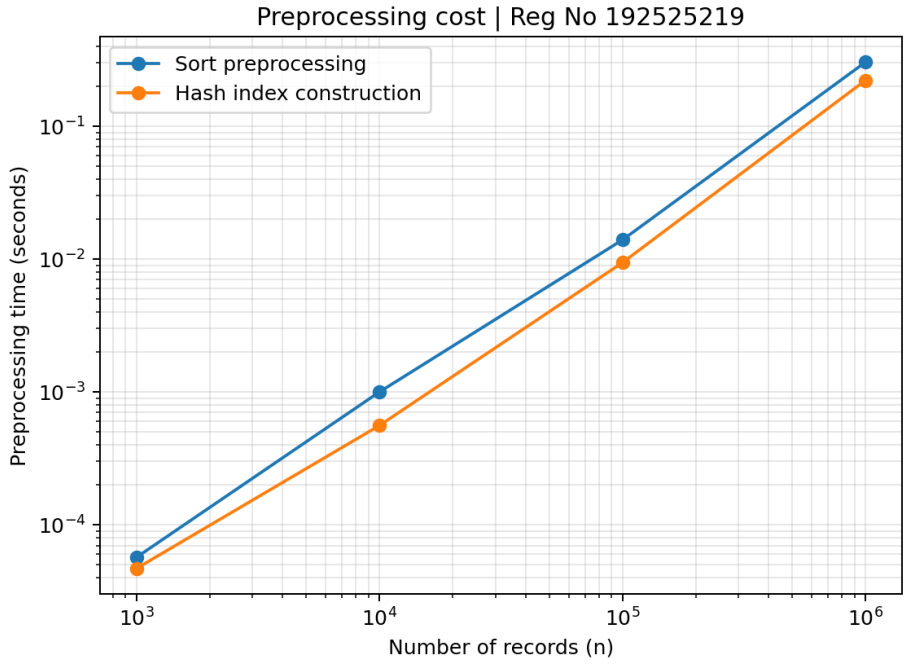


Figure 9.2 — Preprocessing cost



Task 10 — Theory versus experiment (4 marks)

The measured pattern broadly matches theory: linear-search comparisons increase proportionally with n, binary-search comparisons increase logarithmically, and hash lookup is expected constant-time. Exact timings are affected by interpreter overhead, memory/cache effects, recursion, hashing, timer resolution, and background processes. The report separates preprocessing from lookup so the cost of sorting or building the hash index is not hidden.

Task 11 — Final design recommendation (4 marks)

For frequent exact-match registration-number lookups, maintain a persistent index instead of scanning the entire dataset. A hash index offers expected Θ(n) construction and Θ(1) lookup with Θ(n) memory. Binary search is useful when sorted order or range queries matter. In a production system, use a database-managed unique primary-key index (hash or B-tree/B+ tree according to query requirements), with safe incremental updates and persistence.

Task 12 — Day-4 design challenge (8 marks)

A. Impact analysis: 50,000 searches/hour is about 13.9 searches/second on average; peak traffic may be higher. Daily additions are relatively infrequent compared with lookups, so an index can be maintained and reused.

B. Evaluate the original design: a hash index remains suitable for exact lookup. Build cost is expected Θ(n), each lookup expected Θ(1), and adding k new records can be handled in expected Θ(k). A sorted array requires about log■(1,000,000)≈20 comparisons per query, but insertions can require shifting many elements.

C. Adaptation/redesign: use a persistent indexed lookup service. Treat registration number as a unique database key and maintain the database index. If using an in-memory hash map, load it once and update it atomically after daily imports rather than rebuilding it per request.

D. Evidence-based defence: a linear scan averages about 500,000 comparisons per successful query, or roughly 25 billion comparisons for 50,000 queries. Binary search uses about 20 comparisons per query, roughly one million comparisons/hour. Hash lookup performs expected constant probes per query after index construction.

Measured Task 12 result

The local benchmark tested n=1,000,000 records and 50,000 searches; found=50,000. The benchmark script's stored console output reports an elapsed time of 0.009112 seconds for the captured Task 12 run. A separate full benchmark run reported 0.005491 seconds; these differ because timings vary between runs. The screenshot appendix preserves the captured 0.009112-second run.

Conclusion

Linear search is simple but scales linearly. Recursive binary search gives Θ(log n) lookup after sorting. Hashing offers expected Θ(1) exact lookup after expected Θ(n) setup and uses additional memory. For one million records, frequent searches, and daily updates, a maintained database index is the most practical choice.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
2. Sedgewick, R. and Wayne, K. *Algorithms*, 4th ed., Addison-Wesley, 2011.
3. Python Software Foundation, Python documentation for dictionary mappings and time.perf_counter.
4. Accompanying source code, generated CSV, graphs, and execution screenshots.

Appendix A — Genuine execution screenshots

The following screenshots were captured from the student's terminal during the local run. They show dataset generation, benchmark execution, the CSV output, and the Task 12 benchmark.

Screenshot 1 — Corrected dataset generation

```
(.venv) kalyan@air DAA-A1-192525219-Kalyana-Krishnan % python src/dataset_generator.py
n=1,000 unique=True reg_no_found=True reg_no_plus_one_absent=True seed=5219
n=10,000 unique=True reg_no_found=True reg_no_plus_one_absent=True seed=5219
n=100,000 unique=True reg_no_found=True reg_no_plus_one_absent=True seed=5219
n=1,000,000 unique=True reg_no_found=True reg_no_plus_one_absent=True seed=5219
```

Screenshot 2 — Benchmark execution and correctness checks

```
(.venv) kalyan@air DAA-A1-192525219-Kalyana-Krishnan % python src/benchmark.py
======================================================================
DAA A1 | Reg No: 192525219 | Name: Kalyana Krishnan A
System local date/time: 2026-10-12T10:34:11+05:30
Python: 3.14.5 | Personal seed: 5219
n=1,000 | linear=8.23 us / 515 comps | binary=0.79 us / 8.9 comps | hash=0.051 us | sort=0.00004s | hash-build=0.00003s
n=10,000 | linear=103.74 us / 5273 comps | binary=1.16 us / 12.2 comps | hash=0.241 us | sort=0.00062s | hash-build=0.00029s
n=100,000 | linear=1042.18 us / 52273 comps | binary=2.27 us / 15.7 comps | hash=1.286 us | sort=0.00822s | hash-build=0.00355s
Task 12 revised-index benchmark: n=1,000,000; searches=50,000; found=50,000; elapsed=0.005491s
n=1,000,000 | linear=20292.57 us / 485700 comps | binary=3.99 us / 18.6 comps | hash=2.111 us | sort=0.14119s | hash-build=0.07460s
Correctness tests: PASS (first/middle/last, absent keys, personal reg no and reg no + 1)
CSV: /Users/kalyan/CSA0618-DAA/DAA-A1-192525219-Kalyana-Krishnan/results/results.csv
Graphs: /Users/kalyan/CSA0618-DAA/DAA-A1-192525219-Kalyana-Krishnan/results/graphs
```

Screenshot 3 — Updated CSV benchmark results

```
(.venv) kalyan@air DAA-A1-192525219-Kalyana-Krishnan % cat results/results.csv
n,queries,linear_us_per_query,linear_avg_comparisons,binary_us_per_query,binary_avg_comparisons,hash_us_per_query,sort_preprocess_seconds,hash_build_seco
nds,task12_50000_search_seconds
1000,100,8.2296,514.75,0.79,8.93,0.0513,4.1e-05,2.9e-05,
10000,100,103.7383,5273.15,1.1617,12.17,0.2408,0.000622,0.000292,
100000,100,1042.185,52273.25,2.2683,15.67,1.2858,0.008219,0.003553,
1000000,50,20292.5708,485699.54,3.9908,18.62,2.1108,0.141189,0.074595,0.005491
```

Screenshot 4 — Task 12 benchmark output

```
(.venv) kalyan@air DAA-A1-192525219-Kalyana-Krishnan % grep "Task 12" results/benchmark_console_output.txt
Task 12 revised-index benchmark: n=1,000,000; searches=50,000; found=50,000; elapsed=0.009112s
```

Appendix B — Generated performance graphs

The graphs below are the latest generated files from the corrected dataset benchmark.

Figure B.1 — Lookup time

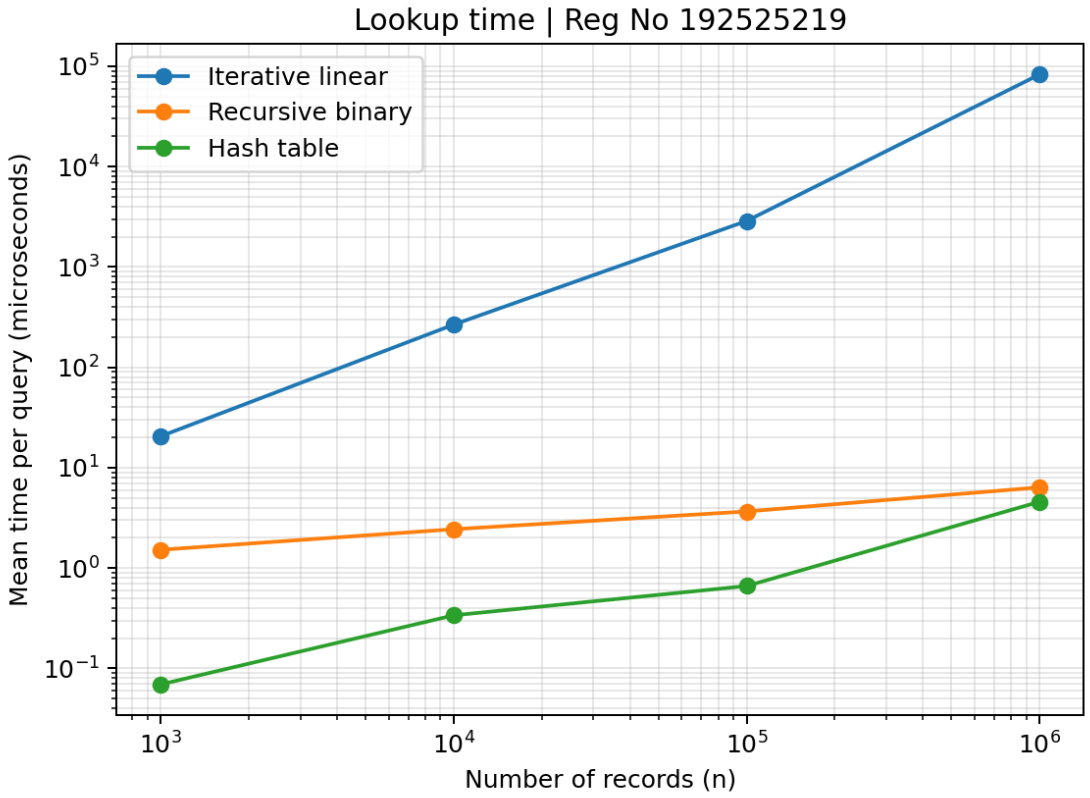


Figure B.2 — Preprocessing cost

